

PROLOG PROGRAMS

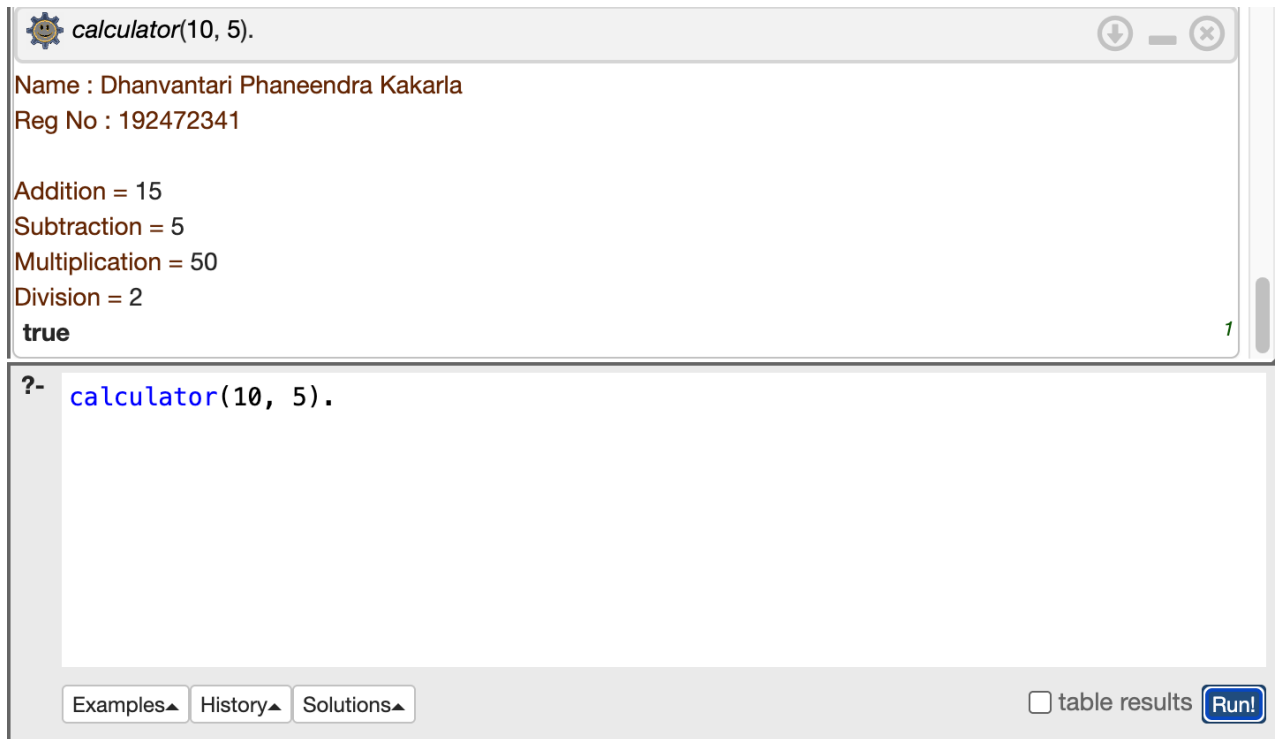
Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

1. BASIC CALCULATOR

Program:

```
calculator(A, B) :-  
    Sum is A + B,  
    Difference is A - B,  
    Product is A * B,  
    Division is A / B,  
    write('Name : Dhanvantari Phaneendra Kakarla, Reg No: 192472341 calculator'), nl,  
    write('Addition      = '), write(Sum), nl,  
    write('Subtraction   = '), write(Difference), nl,  
    write('Multiplication = '), write(Product), nl,  
    write('Division       = '), write(Division), nl.
```

OUTPUT:



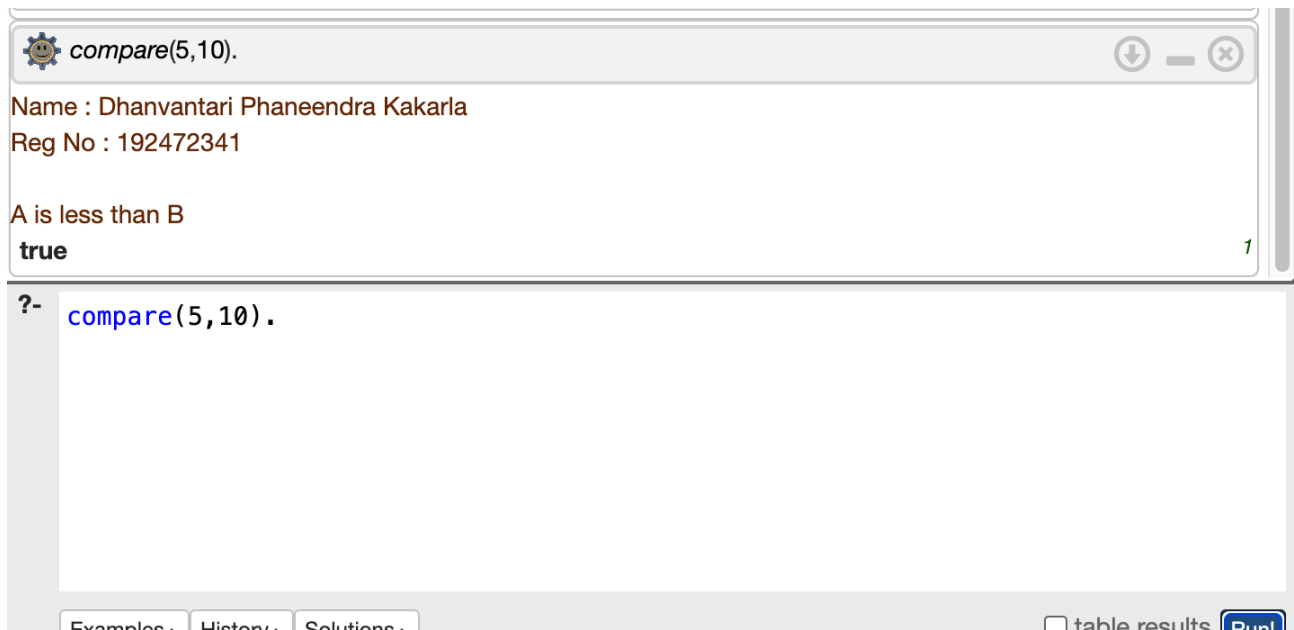
```
calculator(10, 5).  
Name : Dhanvantari Phaneendra Kakarla  
Reg No : 192472341  
  
Addition = 15  
Subtraction = 5  
Multiplication = 50  
Division = 2  
true  
  
?- calculator(10, 5).
```

2. ALL COMPARISON

Program:

```
compare(A, B) :-  
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    (A > B ->  
        write('A is greater than B')  
    ;  
        A < B ->  
        write('A is less than B')  
    ;  
        write('A is equal to B')  
    ).
```

OUTPUT:



The screenshot shows a Prolog IDE window titled "compare(5,10)." with standard window controls. The output area displays the following text:

```
Name : Dhanvantari Phaneendra Kakarla  
Reg No : 192472341  
  
A is less than B  
true
```

At the bottom of the window, there is a prompt "?- compare(5,10)." and a footer bar containing tabs for "Examples", "History", and "Solutions", along with a "table results" button and a "Run" button.

3. VOTE ELIGIBILITY

Program:

```
vote :-  
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Enter your age: '),  
    read(Age),  
    (Age >= 18 ->  
        write('You are eligible to vote.')  
    ;  
        write('You are not eligible to vote.')  
    ),  
    nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Enter your age:

19

You are eligible to vote.

true

1

?- vote.

Examples: History: Solutions:

table results Run

4. CELSIUS TO FAHRENHEIT

Program:

```
celsius_to_fahrenheit :-  
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Enter temperature in Celsius: '),  
    read(C),  
    F is (C * 9 / 5) + 32,  
    write('Temperature in Fahrenheit = '),  
    write(F),  
    nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Enter temperature in Celsius:

67

Temperature in Fahrenheit = 152.6

true

1

?- celsius_to_fahrenheit.

Examples▲

History▲

Solutions▲

☐ table results

Run!

5. STUDENT RESULT

Program:

```
result(M1, M2, M3) :-  
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Subject 1 Mark: '), write(M1), nl,  
    write('Subject 2 Mark: '), write(M2), nl,  
    write('Subject 3 Mark: '), write(M3), nl,  
    nl,  
    (M1 >= 40, M2 >= 40, M3 >= 40 ->  
        write('Result: PASS')  
    ;  
        write('Result: FAIL')  
    ),  
    nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Subject 1 Mark: 75
Subject 2 Mark: 60
Subject 3 Mark: 90

Result: PASS

true

1

?- `result(75,60,90).`

Examples▲

History▲

Solutions▲

☐ table results

Run!

6. AVERAGE OF FIVE NUMBERS

Program:

```
average :-
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    write('Enter first number: '), read(A),
    write('Enter second number: '), read(B),
    write('Enter third number: '), read(C),
    write('Enter fourth number: '), read(D),
    write('Enter fifth number: '), read(E),
    Sum is A + B + C + D + E,
    Avg is Sum / 5,
    nl,
    write('Average = '), write(Avg), nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla

Reg No : 192472341

Enter first number:

12

Enter second number:

13

Enter third number:

14

Enter fourth number:

16

Enter fifth number:

17

Average = 14.4

true

1

?- average.

Examples▲

History▲

Solutions▲

☐ table results

Run!

7. MULTIPLICATION TABLE

Program:

```
table(N, I) :-  
    I <= 10,  
    Result is N * I,  
    write(N), write(' x '), write(I),  
    write(' = '), write(Result), nl,  
    I1 is I + 1,  
    table(N, I1).  
  
table(N) :-  
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Enter a number: '),  
    read(N),  
    table(N, 1).
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Enter a number:

6

6 x 1 = 6
6 x 2 = 12
6 x 3 = 18
6 x 4 = 24
6 x 5 = 30
6 x 6 = 36
6 x 7 = 42
6 x 8 = 48
6 x 9 = 54
6 x 10 = 60
false

?- table(6).

Examples▲ History▲ Solutions▲

☐ table results **Run!**

8. STUDENT-TEACHER-SUBJECT RELATION

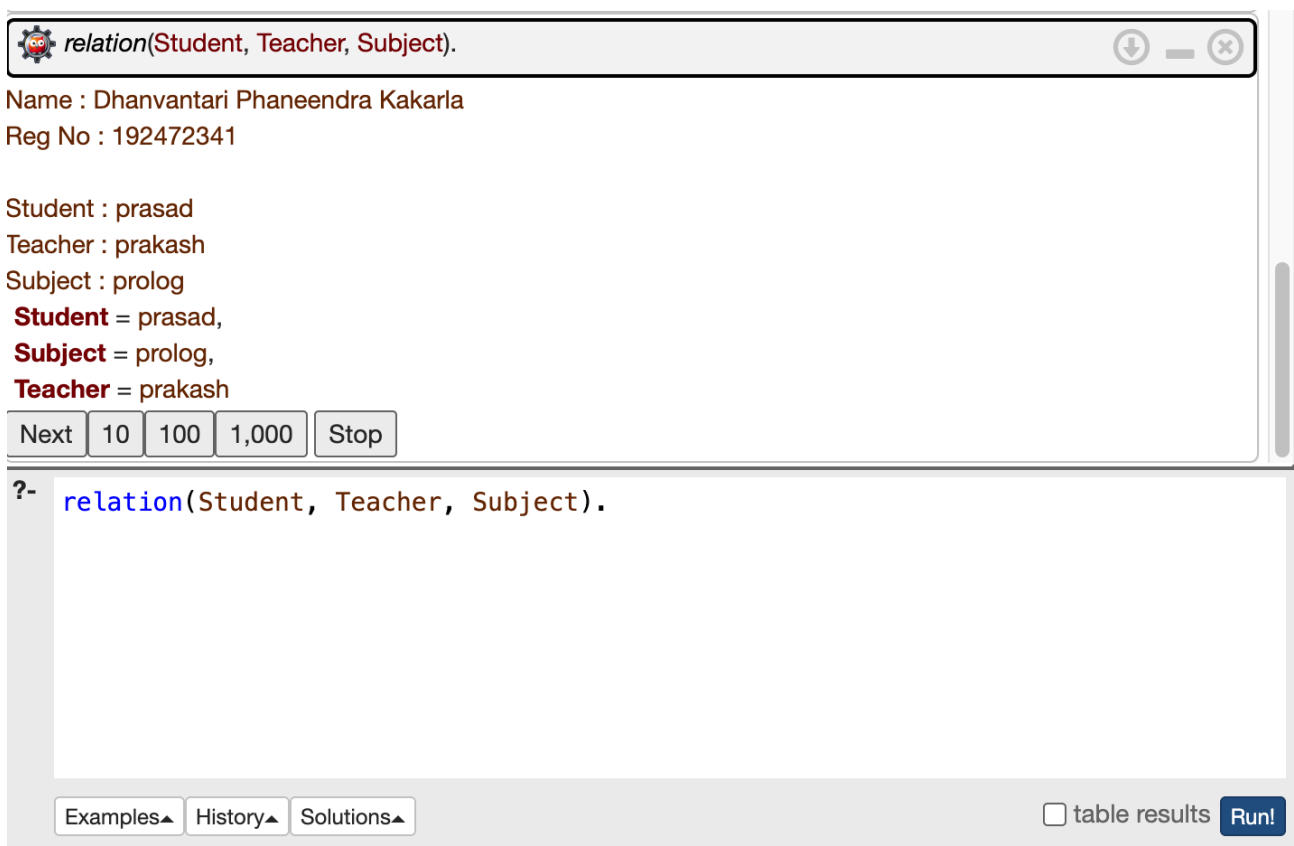
Program:

```
studies(prasad, prolog).
studies(raju, dbms).
studies(sita, python).

% Teacher and the subject they teach
teaches(prakash, prolog).
teaches(ravi, dbms).
teaches(anu, python).

% Display student, teacher and subject
relation(Student, Teacher, Subject) :-
    studies(Student, Subject),
    teaches(Teacher, Subject),
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    write('Student : '), write(Student), nl,
    write('Teacher : '), write(Teacher), nl,
    write('Subject : '), write(Subject), nl.
```

OUTPUT:



The screenshot shows a Prolog IDE window titled "relation(Student, Teacher, Subject)." with standard window controls. The output area displays the following text:

```
Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Student : prasad
Teacher : prakash
Subject : prolog
Student = prasad,
Subject = prolog,
Teacher = prakash
```

Below the output, there are buttons for "Next", "10", "100", "1,000", and "Stop". At the bottom of the IDE, there is a search bar with "?- relation(Student, Teacher, Subject).", a "table results" checkbox, and a "Run!" button. Navigation buttons "Examples▲", "History▲", and "Solutions▲" are also present.

9. FAMILY RELATION

Program:

```
parent(john, mary).
parent(john, peter).
parent(susan, mary).
parent(susan, peter).
parent(peter, tom).
parent(peter, anna).
parent(lisa, tom).
parent(lisa, anna).
```

```
% Male and Female
male(john).
male(peter).
male(tom).
female(susan).
female(mary).
female(lisa).
female(anna).
```

```
% Father
father(X, Y) :-
    male(X),
    parent(X, Y).
```

```
% Mother
mother(X, Y) :-
    female(X),
    parent(X, Y).
```

```
% Child
child(X, Y) :-
    parent(Y, X).
```

```
% Son
son(X, Y) :-
    male(X),
    parent(Y, X).
```

```
% Daughter
daughter(X, Y) :-
    female(X),
    parent(Y, X).
```

```
% Brother
brother(X, Y) :-
    male(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.
```

```
% Sister
sister(X, Y) :-
    female(X),
    parent(P, X),
    parent(P, Y),
    X \= Y.
```

```
% Grandfather
grandfather(X, Y) :-
    male(X),
    parent(X, Z),
```

```

parent(Z, Y).

% Grandmother
grandmother(X, Y) :-
    female(X),
    parent(X, Z),
    parent(Z, Y).

% Grandchild
grandchild(X, Y) :-
    parent(Y, Z),
    parent(Z, X).

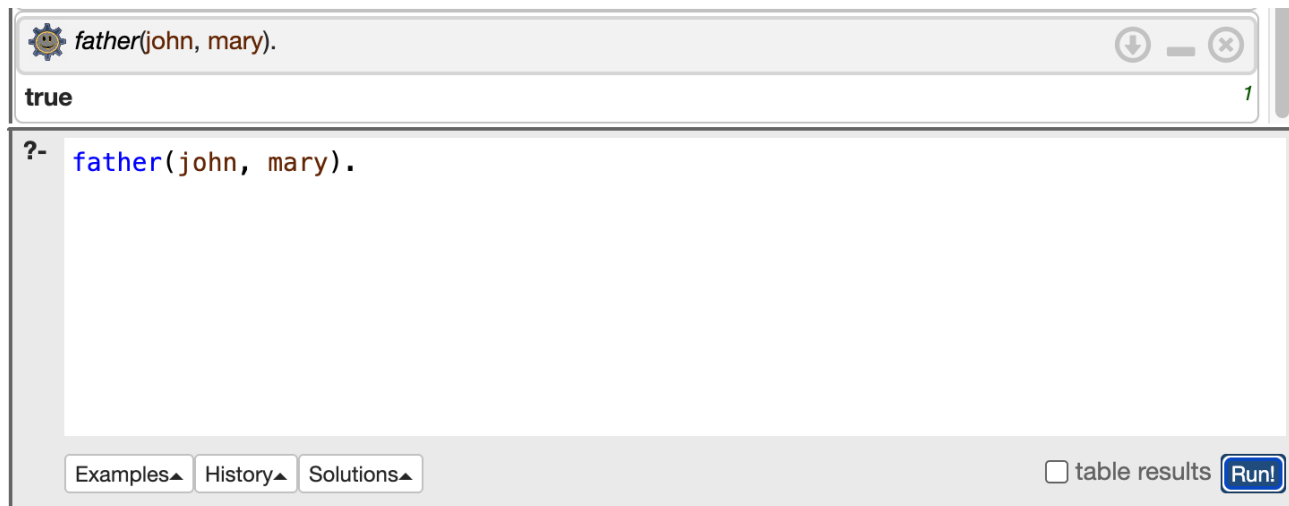
% Uncle
uncle(X, Y) :-
    male(X),
    parent(P, Y),
    brother(X, P).

% Aunt
aunt(X, Y) :-
    female(X),
    parent(P, Y),
    sister(X, P).

% Display student details
show_details :-

```

OUTPUT:



The screenshot shows a Prolog IDE window with a title bar containing a gear icon, the text `father(john, mary).`, and standard window controls. The main area displays the query `?- father(john, mary).` and the result `true`. At the bottom, there are buttons for `Examples▲`, `History▲`, and `Solutions▲`, along with a checkbox for `table results` and a `Run!` button.

10. DIET RECOMMENDATION BASED ON DISEASE

Program:

```
diet(diabetes) :-  
    write('Recommended Diet: Vegetables, whole grains and low-sugar foods. '), nl.  
  
diet(obesity) :-  
    write('Recommended Diet: Fruits, vegetables and low-fat foods. '), nl.  
  
diet(hypertension) :-  
    write('Recommended Diet: Low-salt foods, fruits and vegetables. '), nl.  
  
diet(anemia) :-  
    write('Recommended Diet: Iron-rich foods such as spinach, beans and lentils. '), nl.  
  
diet(ulcer) :-  
    write('Recommended Diet: Soft foods and avoid spicy foods. '), nl.  
  
diet(_) :-  
    write('Disease not found in the system. '), nl.  
  
start :-  
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Enter disease: '),  
    read(Disease),  
    diet(Disease).
```

OUTPUT:

 start.

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Enter disease:

Recommended Diet: Fruits, vegetables and low-fat foods.

true

?- start.

☐ table results

11. FORWARD / BACKWARD CHAINING

Program:

```
bird(parrot).
has_feathers(parrot).

% Rules
animal(X) :-
    bird(X).

can_fly(X) :-
    bird(X),
    has_feathers(X).

% Display details
start :-
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    write('Forward / Backward Chaining'), nl,
    nl,
    write('Bird: parrot'), nl,
    write('Feathers: yes'), nl,
    write('Animal: '),
    (animal(parrot) -> write('Yes') ; write('No')),
    nl,
    write('Can Fly: '),
    (can_fly(parrot) -> write('Yes') ; write('No')),
    nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Forward / Backward Chaining

Bird: parrot
Feathers: yes
Animal: Yes
Can Fly: Yes
true

1

?- **start.**

Examples▲ History▲ Solutions▲

☐ table results

Run!

12. MONKEY AND BANANA PROBLEM

Program:

```
monkey(at_door).
banana(on_ceiling).
box(in_room).

% Monkey can move to the box
move_monkey :-
    monkey(at_door),
    write('Monkey moves to the box.'), nl.

% Monkey pushes the box under the banana
push_box :-
    write('Monkey pushes the box under the banana.'), nl.

% Monkey climbs the box
climb_box :-
    write('Monkey climbs on the box.'), nl.

% Monkey gets the banana
get_banana :-
    banana(on_ceiling),
    write('Monkey gets the banana.'), nl.

% Main program
start :-
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    move_monkey,
    push_box,
    climb_box,
    get_banana,
    nl,
    write('Monkey has the banana!'), nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla

Reg No : 192472341

Monkey moves to the box.

Monkey pushes the box under the banana.

Monkey climbs on the box.

Monkey gets the banana.

Monkey has the banana!

true

1

?- **start.**

Examples▲

History▲

Solutions▲

☐ table results

Run!

13. FRUIT AND COLOR

Program:

```
fruit(apple, red).  
fruit(banana, yellow).  
fruit(orange, orange).  
fruit(grapes, green).  
fruit(mango, yellow).
```

```
start :-  
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Fruit and Color:'), nl,  
    fruit(Fruit, Color),  
    write(Fruit),  
    write(' -> '),  
    write(Color),  
    nl,  
    fail.
```

start.

OUTPUT:

```
Name : Dhanvantari Phaneendra Kakarla  
Reg No : 192472341
```

```
Fruit and Color:  
apple -> red  
banana -> yellow  
orange -> orange  
grapes -> green  
mango -> yellow  
true
```

1

?- start.

Examples▲ History▲ Solutions▲

☐ table results

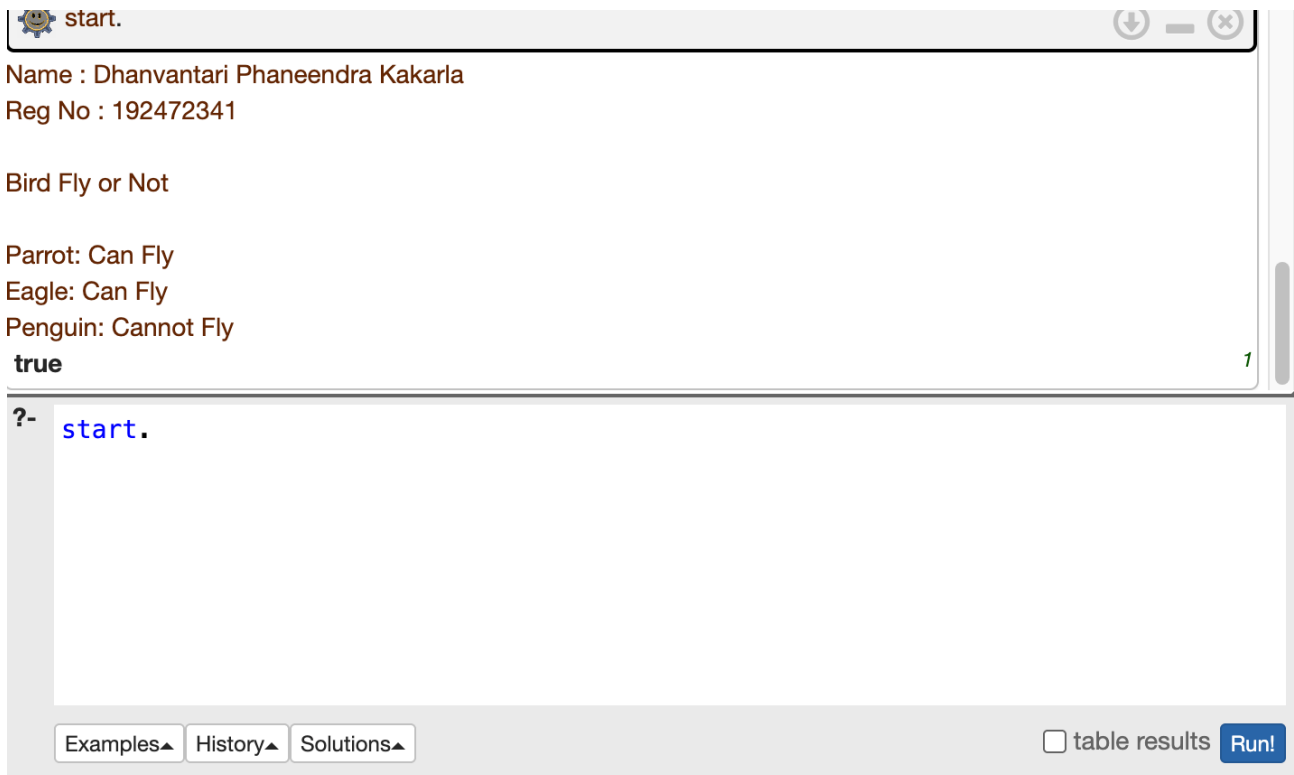
Run!

14. BIRD FLY OR NOT

Program:

```
can_fly(X) :-  
    bird(X),  
    fly(X).  
  
cannot_fly(X) :-  
    bird(X),  
    not_fly(X).  
  
% Display details  
start :-  
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Bird Fly or Not'), nl,  
    nl,  
    write('Parrot: '),  
    (can_fly(parrot) -> write('Can Fly') ; write('Cannot Fly')),  
    nl,  
    write('Eagle: '),  
    (can_fly(eagle) -> write('Can Fly') ; write('Cannot Fly')),  
    nl,  
    write('Penguin: '),  
    (can_fly(penguin) -> write('Can Fly') ; write('Cannot Fly')),  
    nl.
```

OUTPUT:



The screenshot shows a Prolog IDE interface. The top window, titled 'start.', displays the output of the program. The output is as follows:

```
Name : Dhanvantari Phaneendra Kakarla  
Reg No : 192472341  
  
Bird Fly or Not  
  
Parrot: Can Fly  
Eagle: Can Fly  
Penguin: Cannot Fly  
true
```

The bottom window shows the input prompt '?-' followed by the command 'start.'. At the bottom of the IDE, there are buttons for 'Examples▲', 'History▲', and 'Solutions▲'. On the right side, there is a checkbox for 'table results' and a 'Run!' button.

15. GRADE FINDER

Program:

```
grade(Mark, 'A') :-  
    Mark >= 90.  
  
grade(Mark, 'B') :-  
    Mark >= 75,  
    Mark < 90.  
  
grade(Mark, 'C') :-  
    Mark >= 60,  
    Mark < 75.  
  
grade(Mark, 'D') :-  
    Mark >= 50,  
    Mark < 60.  
  
grade(Mark, 'F') :-  
    Mark < 50.  
  
% Main Program  
main :-  
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('Enter Marks: '),  
    read(Mark),  
    grade(Mark, Grade),  
    write('Grade: '),  
    write(Grade),  
    nl.
```

OUTPUT:

main.

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Enter Marks:
90

Grade: A
true

Next 10 100 1,000 Stop

?- main.

Examples History Solutions table results Run!

16. EB BILL CALCULATION

Program:

```
bill(Units, Amount) :-  
    Units <= 100,  
    Amount is Units * 2.  
  
bill(Units, Amount) :-  
    Units > 100,  
    Units <= 200,  
    Amount is (100 * 2) + ((Units - 100) * 3).  
  
bill(Units, Amount) :-  
    Units > 200,  
    Amount is (100 * 2) + (100 * 3) + ((Units - 200) * 5).  
  
% Main Program  
main :-  
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,  
    write('Reg No : 192472341'), nl,  
    nl,  
    write('EB BILL CALCULATION'), nl,  
    nl,  
    write('Enter Units Consumed: '),  
    read(Units),  
    bill(Units, Amount),  
    write('Units Consumed : '), write(Units), nl,  
    write('EB Bill Amount : Rs. '), write(Amount), nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

EB BILL CALCULATION

Enter Units Consumed:

89

Units Consumed : 89

EB Bill Amount : Rs.178

true

Next

10

100

1,000

Stop

?-

main.

Examples▲

History▲

Solutions▲

☐ table results

Run!

17. STUDENT DATABASE

Program:

```
student(101, 'Ravi', cse).
student(102, 'Arun', ece).
student(103, 'Priya', cse).
student(104, 'Kiran', aiml).

% Main Program
main :-
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    write('STUDENT DATABASE'), nl,
    nl,
    write('Student Details:'), nl,
    student(Roll, Name, Department),
    write('Roll No: '), write(Roll),
    write(' Name: '), write(Name),
    write(' Department: '), write(Department), nl,
    fail.

main.
```

OUTPUT:

```
Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

STUDENT DATABASE

Student Details:
Roll No: 101 Name: Ravi Department: cse
Roll No: 102 Name: Arun Department: ece
Roll No: 103 Name: Priya Department: cse
Roll No: 104 Name: Kiran Department: aiml
true
```

?- `main.`

Examples▲ History▲ Solutions▲

☐ table results **Run!**

18. PRINT NUMBERS 1 TO 10

Program:

```
print_numbers(10).

print_numbers(N) :-
    N < 10,
    M is N + 1,
    write(M), nl,
    print_numbers(M).

% Main Program
main :-
    write('Name   : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    write('Numbers from 1 to 10:'), nl,
    write(1), nl,
    print_numbers(1).
```

OUTPUT:

The screenshot shows a Prolog IDE window titled "main.". The output area displays the following text:

```
Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

Numbers from 1 to 10:
1
2
3
4
5
6
7
8
9
10
true
```

At the bottom of the output area, there are buttons for "Next", "10", "100", "1,000", and "Stop". A green "1" is visible on the right side of the output area.

The input area shows a prompt "?- " followed by the text "main.". At the bottom of the IDE, there are buttons for "Examples▲", "History▲", and "Solutions▲". On the right side, there is a checkbox labeled "table results" and a blue button labeled "Run!".

19. N-QUEENS PROBLEM

Program:

```
safe([]).

safe([Q|Queens]) :-
    safe(Queens),
    not_attack(Q, Queens, 1).

not_attack(_, [], _).

not_attack(Q, [Q1|Queens], Distance) :-
    Q \= Q1,
    abs(Q - Q1) \= Distance,
    D2 is Distance + 1,
    not_attack(Q, Queens, D2).

% Generate a solution
queens(N, Solution) :-
    length(Solution, N),
    permutation_list(Solution),
    safe(Solution).

permutation_list([]).

permutation_list(List) :-
    length(List, N),
    num_list(1, N, Numbers),
    permutation(Numbers, List).

num_list(Start, End, []) :-
    Start > End.

num_list(Start, End, [Start|Rest]) :-
    Start <= End,
    Next is Start + 1,
    num_list(Next, End, Rest).

% Main Program
main :-
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    write('N-Queens Problem'), nl,
    nl,
    write('Solution for 4 Queens:'), nl,
    queens(4, Solution),
    write(Solution), nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla

Reg No : 192472341

N-Queens Problem

Solution for 4 Queens:

[2, 4, 1, 3]

true

1

Next

10

100

1,000

Stop

?-

main.

Examples▲

History▲

Solutions▲

☐ table results

Run!

20. REVERSE A NUMBER

Program:

```
reverse_number(0, 0).

reverse_number(N, Reverse) :-
    N > 0,
    reverse_helper(N, 0, Reverse).

reverse_helper(0, Result, Result).

reverse_helper(N, Result, Reverse) :-
    N > 0,
    Digit is N mod 10,
    NewResult is Result * 10 + Digit,
    NewN is N // 10,
    reverse_helper(NewN, NewResult, Reverse).

% Main Program
main :-
    write('Name : Dhanvantari Phaneendra Kakarla'), nl,
    write('Reg No : 192472341'), nl,
    nl,
    write('REVERSE A NUMBER'), nl,
    nl,
    write('Enter a number: '),
    read(Number),
    reverse_number(Number, Reverse),
    write('Original Number : '), write(Number), nl,
    write('Reversed Number : '), write(Reverse), nl.
```

OUTPUT:

Name : Dhanvantari Phaneendra Kakarla
Reg No : 192472341

REVERSE A NUMBER

Enter a number:

69

Original Number : 69

Reversed Number : 96

true

1

Next 10 100 1,000 Stop

?- main.

Examples▲ History▲ Solutions▲

☐ table results Run!